JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AbstractMap<K,V>

java.lang.Object
    java.util.AbstractMap<K,V>

Type Parameters:

K - the type of keys maintained by this map

V - the type of mapped values

All Implemented Interfaces:

Map<K,V>

Direct Known Subclasses:

ConcurrentHashMap, ConcurrentSkipListMap, EnumMap, HashMap, IdentityHashMap, TreeMap, WeakHashMap

---

```
public abstract class AbstractMap<K,V>
extends Object
implements Map<K,V>
```

This class provides a skeletal implementation of the Map interface, to minimize the effort required to implement this interface.

To implement an unmodifiable map, the programmer needs only to extend this class and provide an implementation for the entrySet method, which returns a set-view of the map's mappings. Typically, the returned set will, in turn, be implemented atop AbstractSet. This set should not support the add or remove methods, and its iterator should not support the remove method.

To implement a modifiable map, the programmer must additionally override this class's put method (which otherwise throws an UnsupportedOperationException), and the iterator returned by entrySet().iterator() must additionally implement its remove method.

The programmer should generally provide a void (no argument) and map constructor, as per the recommendation in the Map interface specification.

The documentation for each non-abstract method in this class describes its implementation in detail. Each of these methods may be overridden if the map being implemented admits a more efficient implementation.

This class is a member of the Java Collections Framework.

Since:

1.2

See Also:

Map, Collection

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	AbstractMap.SimpleEntry<V>	An Entry maintaining a key and a value.
static class	AbstractMap.SimpleImmutaV>	An unmodifiable Entry maintaining a key and a value.

Nested classes/interfaces declared in interface Map

Map.Entry<K,V>

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	AbstractMap()	Sole constructor.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	clear()	Removes all of the mappings from this map (optional operation).
protected Object	clone()	Returns a shallow copy of this AbstractMap instance: the keys and values themselves are not cloned.
boolean	containsKey(Object key)	

		Returns `true` if this map contains a mapping for the specified key.
`boolean`	**`containsValue`**`(Object value)`	Returns `true` if this map maps one or more keys to the specified value.
`boolean`	**`equals`**`(Object o)`	Compares the specified object with this map for equality.
`V`	**`get`**`(Object key)`	Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.
`int`	**`hashCode`**`()`	Returns the hash code value for this map.
`boolean`	**`isEmpty`**`()`	Returns `true` if this map contains no key-value mappings.
`Set<K>`	**`keySet`**`()`	Returns a `Set` view of the keys contained in this map.
`V`	**`put`**`(K key, V value)`	Associates the specified value with the specified key in this map (optional operation).
`void`	**`putAll`**`(Map<? extends K, ? extends V> m)`	Copies all of the mappings from the specified map to this map (optional operation).
`V`	**`remove`**`(Object key)`	Removes the mapping for a key from this map if it is present (optional operation).
`int`	**`size`**`()`	

		Returns the number of key-value mappings in this map.
String	**toString**()	Returns a string representation of this map.
Collection<V>	**values**()	Returns a Collection view of the values contained in this map.

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface Map

compute, computeIfAbsent, computeIfPresent, entrySet, forEach, getOrDefault, merge, putIfAbsent, remove, replace, replace, replaceAll

Constructor Details

AbstractMap

```
protected AbstractMap()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Details

size

```
public int size()
```

Returns the number of key-value mappings in this map. If the map contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Specified by:

`size` in interface `Map<K,V>`

Implementation Requirements:

This implementation returns `entrySet().size()`.

Returns:

the number of key-value mappings in this map

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this map contains no key-value mappings.

Specified by:

`isEmpty` in interface `Map<K,V>`

Implementation Requirements:

This implementation returns `size() == 0`.

Returns:

`true` if this map contains no key-value mappings

containsValue

```
public boolean containsValue(Object value)
```

Returns `true` if this map maps one or more keys to the specified value. More formally, returns `true` if and only if this map contains at least one mapping to a value `v` such that `Objects.equals(value, v)`. This operation will probably require time linear in the map size for most implementations of the `Map` interface.

Specified by:

`containsValue` in interface `Map<K,V>`

Implementation Requirements:

This implementation iterates over `entrySet()` searching for an entry with the specified value. If such an entry is found, `true` is returned. If the iteration terminates without finding such an entry, `false` is returned. Note that this implementation requires linear time in the size of the map.

Parameters:

`value` - value whose presence in this map is to be tested

Returns:

`true` if this map maps one or more keys to the specified value

Throws:

`ClassCastException` - if the value is of an inappropriate type for this map (optional)

`NullPointerException` - if the specified value is null and this map does not permit null values (optional)

containsKey

```
public boolean containsKey(Object key)
```

Returns `true` if this map contains a mapping for the specified key. More formally, returns `true` if and only if this map contains a mapping for a key `k` such that `Objects.equals(key, k)`. (There can be at most one such mapping.)

Specified by:

`containsKey` in interface `Map<K,V>`

Implementation Requirements:

This implementation iterates over `entrySet()` searching for an entry with the specified key. If such an entry is found, `true` is returned. If the iteration terminates without finding such an entry, `false` is returned. Note that this implementation requires linear time in the size of the map; many implementations will override this method.

Parameters:

`key` - key whose presence in this map is to be tested

Returns:

`true` if this map contains a mapping for the specified key

Throws:

`ClassCastException` - if the key is of an inappropriate type for this map (optional)

`NullPointerException` - if the specified key is null and this map does not permit null keys (optional)

get

```
public V get(Object key)
```

Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.

More formally, if this map contains a mapping from a key `k` to a value `v` such that `Objects.equals(key, k)`, then this method returns `v`; otherwise it returns `null`. (There can be at most one such mapping.)

If this map permits null values, then a return value of `null` does not *necessarily* indicate that the map contains no mapping for the key; it's also possible that the map explicitly maps the key to `null`. The `containsKey` operation may be used to distinguish these two cases.

Specified by:

`get` in interface `Map<K,V>`

Implementation Requirements:

This implementation iterates over `entrySet()` searching for an entry with the specified key. If such an entry is found, the entry's value is returned. If the iteration terminates without finding such an entry, `null` is returned. Note that this implementation requires linear time in the size of the map; many implementations will override this method.

Parameters:

`key` - the key whose associated value is to be returned

Returns:

the value to which the specified key is mapped, or `null` if this map contains no mapping for the key

Throws:

`ClassCastException` - if the key is of an inappropriate type for this map (optional)

`NullPointerException` - if the specified key is null and this map does not permit null keys (optional)

put

```
public V put(K key,
             V value)
```

Associates the specified value with the specified key in this map (optional operation). If the map previously contained a mapping for the key, the old value is replaced by the specified value. (A map `m` is said to contain a mapping for a key `k` if and only if `m.containsKey(k)` would return `true`.)

Specified by:

`put` in interface `Map<K,V>`

Implementation Requirements:

This implementation always throws an `UnsupportedOperationException`.

Parameters:

`key` - key with which the specified value is to be associated

`value` - value to be associated with the specified key

Returns:

the previous value associated with `key`, or `null` if there was no mapping for `key`. (A `null` return can also indicate that the map previously associated `null` with `key`, if the implementation supports `null` values.)

Throws:

`UnsupportedOperationException` - if the `put` operation is not supported by this map

`ClassCastException` - if the class of the specified key or value prevents it from being stored in this map

`NullPointerException` - if the specified key or value is null and this map does not permit null keys or values

`IllegalArgumentException` - if some property of the specified key or value prevents it from being stored in this map

remove

```
public V remove(Object key)
```

Removes the mapping for a key from this map if it is present (optional operation). More formally, if this map contains a mapping from key `k` to value `v` such that `Objects.equals(key, k)`, that mapping is removed. (The map can contain at most one such mapping.)

Returns the value to which this map previously associated the key, or `null` if the map contained no mapping for the key.

If this map permits null values, then a return value of `null` does not *necessarily* indicate that the map contained no mapping for the key; it's also possible that the map explicitly mapped the key to `null`.

The map will not contain a mapping for the specified key once the call returns.

Specified by:

`remove` in interface `Map<K,V>`

Implementation Requirements:

This implementation iterates over `entrySet()` searching for an entry with the specified key. If such an entry is found, its value is obtained with its `getValue` operation, the entry is removed from the collection (and the backing map) with the iterator's `remove` operation, and the saved value is returned. If the iteration terminates without finding such an entry, `null` is returned. Note that this implementation requires linear time in the size of the map; many implementations will override this method.

Note that this implementation throws an `UnsupportedOperationException` if the `entrySet` iterator does not support the `remove` method and this map contains a mapping for the specified key.

Parameters:

`key` - key whose mapping is to be removed from the map

Returns:

the previous value associated with `key`, or `null` if there was no mapping for `key`.

Throws:

`UnsupportedOperationException` - if the `remove` operation is not supported by this map

`ClassCastException` - if the key is of an inappropriate type for this map (optional)

`NullPointerException` - if the specified key is null and this map does not permit null keys (optional)

putAll

```
public void putAll(Map<? extends K, ? extends V> m)
```

Copies all of the mappings from the specified map to this map (optional operation). The effect of this call is equivalent to that of calling `put(k, v)` on this map once for each mapping from key `k` to value `v` in the specified map. The behavior of this operation is undefined if the specified map is modified while the operation is in progress. If the specified map has a defined encounter order, processing of its mappings generally occurs in that order.

Specified by:

`putAll` in interface `Map<K,V>`

Implementation Requirements:

This implementation iterates over the specified map's `entrySet()` collection, and calls this map's `put` operation once for each entry returned by the iteration.

Note that this implementation throws an `UnsupportedOperationException` if this map does not support the `put` operation and the specified map is nonempty.

Parameters:

`m` - mappings to be stored in this map

Throws:

`UnsupportedOperationException` - if the `putAll` operation is not supported by this map

`ClassCastException` - if the class of a key or value in the specified map prevents it from being stored in this map

`NullPointerException` - if the specified map is null, or if this map does not permit null keys or values, and the specified map contains null keys or values

IllegalArgumentException - if some property of a key or value in the specified map prevents it from being stored in this map

clear

```
public void clear()
```

Removes all of the mappings from this map (optional operation). The map will be empty after this call returns.

Specified by:

clear in interface Map<K,V>

Implementation Requirements:

This implementation calls `entrySet().clear()`.

Note that this implementation throws an `UnsupportedOperationException` if the `entrySet` does not support the `clear` operation.

Throws:

UnsupportedOperationException - if the `clear` operation is not supported by this map

keySet

```
public Set<K> keySet()
```

Returns a Set view of the keys contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

keySet in interface Map<K,V>

Implementation Requirements:

This implementation returns a set that subclasses AbstractSet. The subclass's iterator method returns a "wrapper object" over this map's `entrySet()` iterator. The `size` method delegates to this map's `size` method and the `contains` method delegates to this map's `containsKey` method.

The set is created the first time this method is called, and returned in response to all subsequent calls. No synchronization is performed, so there is a slight chance that multiple calls to this method will not all return the same set.

Returns:

a set view of the keys contained in this map

values

```
public Collection<V> values()
```

Returns a `Collection` view of the values contained in this map. The collection is backed by the map, so changes to the map are reflected in the collection, and vice-versa. If the map is modified while an iteration over the collection is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined. The collection supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Collection.remove`, `removeAll`, `retainAll` and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

`values` in interface `Map<K,V>`

Implementation Requirements:

This implementation returns a collection that subclasses `AbstractCollection`. The subclass's iterator method returns a "wrapper object" over this map's `entrySet()` iterator. The `size` method delegates to this map's `size` method and the `contains` method delegates to this map's `containsValue` method.

The collection is created the first time this method is called, and returned in response to all subsequent calls. No synchronization is performed, so there is a slight chance that multiple calls to this method will not all return the same collection.

Returns:

a collection view of the values contained in this map

equals

```
public boolean equals(Object o)
```

Compares the specified object with this map for equality. Returns `true` if the given object is also a map and the two maps represent the same mappings. More formally, two maps `m1` and `m2` represent the same mappings if `m1.entrySet().equals(m2.entrySet())`. This ensures that the `equals` method works properly across different implementations of the `Map` interface.

Specified by:

`equals` in interface `Map<K,V>`

Overrides:

`equals` in class `Object`

Implementation Requirements:

This implementation first checks if the specified object is this map; if so it returns `true`. Then, it checks if the specified object is a map whose size is identical to the size of this map; if not, it returns `false`. If so, it iterates over this map's `entrySet` collection, and checks that the specified map contains each mapping that this map contains. If the specified map fails to contain such a mapping, `false` is returned. If the iteration completes, `true` is returned.

Parameters:

`o` - object to be compared for equality with this map

Returns:

`true` if the specified object is equal to this map

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns the hash code value for this map. The hash code of a map is defined to be the sum of the hash codes of each entry in the map's `entrySet()` view. This ensures that `m1.equals(m2)` implies that `m1.hashCode()==m2.hashCode()` for any two maps `m1` and `m2`, as required by the general contract of `Object.hashCode()`.

Specified by:

`hashCode` in interface `Map<K,V>`

Overrides:

`hashCode` in class `Object`

Implementation Requirements:

This implementation iterates over `entrySet()`, calling `hashCode()` on each element (entry) in the set, and adding up the results.

Returns:

the hash code value for this map

See Also:

`Map.Entry.hashCode()`, `Object.equals(Object)`, `Set.equals(Object)`

toString

```
public String toString()
```

Returns a string representation of this map. The string representation consists of a list of key-value mappings in the order returned by the map's `entrySet` view's iterator, enclosed in braces (`"{}"`). Adjacent mappings are separated by the characters `", "` (comma and space). Each key-value mapping is rendered as the key followed by an equals sign (`"="`) followed by the associated value. Keys and values are converted to strings as by `String.valueOf(Object)`.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this map

clone

```
protected Object clone()
                throws CloneNotSupportedException
```

Returns a shallow copy of this `AbstractMap` instance: the keys and values themselves are not cloned.

Overrides:

`clone` in class `Object`

Returns:

a shallow copy of this map

Throws:

`CloneNotSupportedException` - if the object's class does not support the `Cloneable` interface. Subclasses that override the `clone` method can also throw this exception to indicate that an instance cannot be cloned.

See Also:

`Cloneable`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.